Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

Samaresh	**PES2UG24CS657**
Gurudath A N	**PES2UG24CS633**

under the guidance of

Prof. Neha Sharma

Assistant Professor

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

Samaresh	**PES2UG24CS657**
Gurudath A N	**PES2UG24CS633**

In partial fulfilment for the completion of forth semester OS Project (UE24CS242B) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature

Prof. Neha Sharma

Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Operating System** has been carried out by us under the guidance of **Prof. Neha Sharma, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester Jan – May 2026.

Samaresh	**PES2UG24CS657**
Gurudath A N	**PES2UG24CS633**

ACKNOWLEDGEMENT

I would like to express my gratitude to Prof. Neha Sharma, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

This project implements a lightweight container runtime along with a kernel module for memory monitoring in Linux. The system consists of a user-space engine that manages container execution and a kernel-level component that tracks memory usage using Resident Set Size (RSS).

Communication between the command-line interface and the supervisor is achieved using UNIX domain sockets. A bounded-buffer logging mechanism is used to safely capture container output. The kernel module enforces both soft and hard memory limits. When the soft limit is exceeded, a warning is generated, and when the hard limit is crossed, the process is terminated.

The project also demonstrates CPU scheduling behavior by varying Linux nice values for different workloads. Overall, this implementation showcases key operating system concepts such as process isolation, inter-process communication, memory management, synchronization, and kernel-user interaction.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	1
2.	PROBLEM DEFINITION	4
3.	METHODOLOGY	8
4.	CODE	12
5.	RESULT AND DISCUSSION	18
6.	OUTPUTS	20
7.	GITHUB LINK	20
REFERENCES/BIBLIOGRAPHY		21
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		21

INTRODUCTION:

Containers are lightweight virtualization techniques that allow multiple isolated processes to run on a single operating system kernel. Unlike virtual machines, containers share the host kernel while maintaining separation in terms of filesystem, processes, and resources.

In modern systems, controlling resource usage is important to prevent a single process from affecting overall system performance. This project focuses on building a simple container runtime with kernel-level memory monitoring.

The system demonstrates how operating systems manage process execution, enforce limits, and provide controlled environments using both user-space and kernel-space components.

PROBLEM DEFINITION :

In systems running multiple processes, unregulated memory usage can lead to instability, performance issues, or crashes. User-space monitoring alone is not sufficient, as it cannot strictly enforce limits.

The key problems addressed in this project are:

- Lack of strict memory limit enforcement
- Poor isolation between workloads
- Inefficient logging and monitoring mechanisms

The objective of this project is to:

- Run multiple containers in isolated environments
- Monitor memory usage at the kernel level
- Enforce soft and hard memory limits
- Provide reliable logging of container activity
- Demonstrate scheduling behavior using nice values

METHODOLOGY:

The system is designed using multiple components working together to provide containerization and monitoring.

The container runtime engine is responsible for creating and managing containers. It uses system calls such as clone() and chroot() to isolate processes.

A supervisor process manages multiple containers and handles communication with the CLI using UNIX domain sockets.

A kernel module monitors memory usage by tracking the Resident Set Size (RSS) of each container process. It enforces limits by generating warnings or terminating processes.

A logging system based on a bounded-buffer approach is used to safely collect container output using a producer-consumer model.

Scheduling behavior is analyzed by assigning different nice values to CPU-intensive workloads and observing their execution.

CODE :

The implementation consists of both user-space and kernel-space components.

The container runtime uses clone() to create isolated processes and chroot() to provide filesystem isolation.

The kernel module periodically checks memory usage (RSS) of registered processes and compares it with configured limits.

The logging system uses a thread-safe bounded buffer where container output is produced and consumed by a logging thread.

The complete source code is available in the project repository.

RESULT AND DISCUSSION:

The system was tested using different workloads to evaluate its behavior.

The memory_hog program exceeded defined memory limits and was successfully detected by the kernel module. Soft limit warnings were generated, and hard limits resulted in process termination.

The cpu_hog program consumed CPU resources but stayed within memory limits, allowing it to run continuously.

The scheduling experiment showed that processes with lower nice values received higher CPU priority.

These results confirm that the system correctly enforces memory limits, maintains isolation, and demonstrates scheduling behavior.

OUTPUTS:

The outputs demonstrate the working of the container runtime and kernel module.

Multiple containers were successfully launched and managed by a single supervisor.

The engine ps command displayed container metadata such as process IDs, states, and limits.

Logging output confirmed that container data was captured correctly using a bounded-buffer mechanism.

The CLI successfully communicated with the supervisor using IPC.

Kernel logs showed soft limit warnings and hard limit enforcement.

Scheduling behavior was observed using different nice values.

The system also demonstrated clean shutdown without leaving zombie processes.

LIST OF FIGURES (Output Screenshots)

Figure No.	Title
6.1	**Multi-container supervision showing two containers running under a single supervisor**

```
samaresh@S-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine supervisor ./rootfs-base
[supervisor] control socket: /tmp/mini_runtime.sock
[supervisor] logging thread started
[supervisor] ready. base-rootfs=./rootfs-base
```

```
[supervisor] launched container id=c1 pid=3558
[supervisor] launched container id=c2 pid=3566
```

```
samaresh@S-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine start c1 ./rootfs-alpha ./memory_hog
sudo ./engine start c2 ./rootfs-beta ./cpu_hog
started container 'c1' pid=3558
started container 'c2' pid=3566
samaresh@S-VirtualBox:~/OS-Jackfruit/boilerplate$
```

6.2	**Container metadata output using engine ps command**

```
samaresh@S-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
                pid=3566   state=exited     soft=40MiB hard=64MiB log=logs/c2.log
                pid=3558   state=running    soft=40MiB hard=64MiB log=logs/c1.log
samaresh@S-VirtualBox:~/OS-Jackfruit/boilerplate$
```

6.4 Logging output demonstrating bounded-buffer mechanism

```
samaresh@S-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine logs c1
allocation=1 chunk=8MB total=8MB
allocation=2 chunk=8MB total=16MB
allocation=3 chunk=8MB total=24MB
allocation=4 chunk=8MB total=32MB
allocation=5 chunk=8MB total=40MB
allocation=6 chunk=8MB total=48MB
allocation=7 chunk=8MB total=56MB
```

6.5 CLI interaction showing container stop command via IPC

```
samaresh@S-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine stop c1
sent SIGTERM to container 'c1' (pid=3558)
```

6.6 Kernel log showing soft memory limit warning

```
[  619.070031] [container_monitor] Registering container=c1 pid=3689 soft=20971520 hard=104857600
[  621.422762] [container_monitor] SOFT LIMIT container=c1 pid=3689 rss=25812992 limit=20971520
```

Kernel log showing hard memory limit

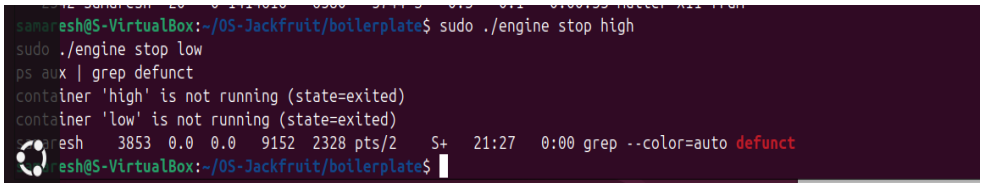
```
[  885.673755] [container_monitor] SOFT LIMIT container=c2 pid=3746 rss=25632768 limit=20971520
[  887.721635] [container_monitor] HARD LIMIT container=c2 pid=3746 rss=42409984 limit=41943040
```

6.7 Scheduling experiment showing concurrent execution of high and low priority containers

```
samaresh@S-VirtualBox:~$ cd OS-Jackfruit/boilerplate
samaresh@S-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine logs low
[sudo] password for samaresh:
cpu_hog alive elapsed=1 accumulator=1941700603874815824
cpu_hog alive elapsed=2 accumulator=13021365196112106549
cpu_hog alive elapsed=3 accumulator=4096392575005596825
cpu_hog alive elapsed=4 accumulator=11111470152421628202
```

```
samaresh@S-VirtualBox:~$ cd OS-Jackfruit/boilerplate
samaresh@S-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine logs high
[sudo] password for samaresh:
cpu_hog alive elapsed=1 accumulator=13286177086777326636
cpu_hog alive elapsed=2 accumulator=4158912309104897708
cpu_hog alive elapsed=3 accumulator=12827295915872998745
cpu_hog alive elapsed=4 accumulator=263500322083757524
```

6.8 **Clean teardown showing no zombie (defunct) processes**



7.GITHUB LINK

are The comprehensive source code repository and version control history for this project accessible via https://github.com/86samaresh/OS-Jackfruit

REFERENCES/BIBLIOGRAPHY :

1. Linux Manual Pages (man clone, man fork, man nice)
2. Linux Kernel Documentation and Specification
3. Operating System Concepts – Abraham Silberschatz
4. Scholarly online repositories and technical documentation on Linux system programming

APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS :

- RSS – Resident Set Size
- IPC – Inter-Process Communication
- LKM – Loadable Kernel Module
- CFS – Completely Fair Scheduler
- CLI – Command Line Interface